alpe FORMACIÓN

DESARROLLO WEB EN ENTORNO CLIENTE

PRÁCTICAS GUIADAS

OBJETOS CON CLASES EN JAVASCRIPT

Seis prácticas con el HTML y el CSS ya realizados: tú solo escribes el JavaScript, siguiendo el enunciado y los pasos de la cabecera de cada script.js



Realizado por Ramón Abramo

ALPE Formación • Santander

Desarrollo de Aplicaciones Web



Índice de contenidos y guía de trabajo

ESTRUCTURA DE LAS PRÁCTICAS, ITINERARIO Y ENLACES RÁPIDOS

Esta hoja de ruta reúne todo lo que vas a hacer.

Pulsa en cualquier tarjeta para saltar a su sección.

TEORÍA Y FUNDAMENTOS

Cómo trabajamos y teoría básica

La carpeta de prácticas, la cabecera de script.js y la sintaxis de clases.

Ir a la teoría (pág. 3)

PRÁCTICA 1 · GUIADA

Garaje de coches

Clases, constructor, métodos y getters.

Ir a la práctica 1 (pág. 6)

PRÁCTICA 2 · GUIADA

Cuenta bancaria segura

Campos privados, getters y errores controlados.

Ir a la práctica 2 (pág. 9)

PRÁCTICA 3 · GUIADA

Figuras geométricas

Herencia, super, polimorfismo y miembros estáticos.

Ir a la práctica 3 (pág. 12)

PRÁCTICA 4 · GUIADA

Biblioteca de libros

Colecciones de objetos, composición y toString().

Ir a la práctica 4 (pág. 15)

PRÁCTICA 5 · GUIADA

Gestor de tareas con JSON

Clases, JSON, toJSON(), static y localStorage.

Ir a la práctica 5 (pág. 18)

PRÁCTICA 6 · RETO FINAL

Tienda con carrito

Reto final: herencia, composición, getters y estáticos.

Ir a la práctica 6 (pág. 21)

FUENTES

Fuentes y Documentación Oficial

Documentación en la que se apoyan las prácticas.

Ir a las fuentes (pág. 24)



Cómo vamos a trabajar

CARPETAS, ARCHIVOS Y LA CABECERA DE SCRIPT.JS



La carpeta de prácticas

Cada práctica tiene su propia carpeta dentro de `practicas`.

Copia la carpeta `practicas` en `C:\laragon\www\js\clases\`.





Qué contiene cada práctica

Archivo	Estado	Tu trabajo
`index.html`	**Ya realizado**	No lo modifiques
`estilos.css`	**Ya realizado**	No lo modifiques
`script.js`	Con la cabecera y los `TODO`	**Es lo único que escribes**



La cabecera de script.js

En la cabecera encontrarás el **enunciado** y la lista de **pasos** a realizar.

Dentro del archivo, cada paso tiene su comentario `TODO PASO n` con las indicaciones.



Cómo vamos a trabajar · La cabecera

ASÍ EMPIEZA CADA SCRIPT.JS

script.js · cabecera de la práctica 1

```
/* ============================================================
   PRÁCTICA 01 · GARAJE DE COCHES
   Tema: Clases, constructor, métodos y getters
   ------------------------------------------------------------
   ENUNCIADO
   Vas a programar un garaje donde se guardan coches.
   Cada coche será un objeto creado con la clase Coche.
   Al rellenar el formulario se crea un coche nuevo y aparece una tarjeta con sus datos.
   Cada tarjeta tiene un botón para sumar 100 km al coche.
   ------------------------------------------------------------
   TODO (PASOS A REALIZAR)
   [ ] PASO 1: Crea la clase Coche
         Con el constructor (marca, modelo, anio, km = 0). Guarda los datos con this.
   [ ] PASO 2: Método describir()
         Devuelve un texto como "Seat León (2019) · 45000 km".
   [ ] PASO 3: Método recorrer(km)
         Suma los km al coche solo si km es mayor que 0. Devuelve this.
   [ ] PASO 4: Getter antiguedad
         Devuelve los años del coche: año actual menos this.anio. Usa new Date().getFullYea
r().
   [ ] PASO 5: Crea el array coches
         Será una constante con un array vacío.
   [ ] PASO 6: Función dibujar()
         Vacía listaCoches y crea una tarjeta por coche con crearTarjeta(). Añade el botón
"+100 km" y actualiza el resumen.
   [ ] PASO 7: Escuchador del formulario
         Al enviar: evita la recarga, valida el año, crea el Coche, lo guarda en el array y
llama a dibujar().
   ------------------------------------------------------------
   NORMAS
   - Tu trabajo es SOLO este archivo (script.js).
   - No modifiques index.html ni estilos.css: ya están realizados.
   - Escribe el código debajo de cada comentario TODO.
   - Marca cada paso con [x] cuando lo termines.
   ============================================================ */
```



Tres reglas de trabajo

1 · Un paso cada vez

Completa un `TODO`, guarda, recarga y comprueba antes de seguir.

2 · Lee los errores

La consola (`F12`) indica el tipo de error, el archivo y la línea.

3 · Escribe tú el código

Consulta el manual de clases, pero no copies una solución.



Teoría básica

LA SINTAXIS DE CLASES QUE NECESITAS

Una **clase** es una plantilla para crear objetos del mismo tipo. Cada objeto creado con `new` es una **instancia**. [1]

Concepto	Sintaxis	Ejemplo
Clase y constructor	`class A { constructor(...) { } }` [2]	`class Coche { constructor(marca) { this.marca = marca; } }`
Instancia	`new A(...)`	`const c = new Coche("Seat");`
Método	`nombre() { ... }`	`describir() { return this.marca; }`
Getter / setter	`get x() { } · set x(v) { }`	`get antiguedad() { ... }`
Campo privado	`#campo`	`#saldo = 0;`
Estático	`static campo · static metodo()`	`static total() { ... }`
Herencia	`class B extends A` y `super(...)`	`class Perro extends Animal`
Error controlado	`throw new Error()` y `try...catch`	`throw new Error("PIN incorrecto");`



Objeto literal frente a clase

Criterio	Objeto literal	Clase
Cuántos objetos	Uno concreto	Todos los que quieras con la misma estructura
Métodos	Una copia por objeto	Compartidos entre las instancias
Privacidad y herencia	No disponibles	`#` y `extends`

> **Fíjate:** esta tabla resume lo esencial. Para ampliarlo, consulta el **Manual de clases en JavaScript** y la carpeta **ejemplos**.



Práctica 1

CLASES, CONSTRUCTOR, MÉTODOS Y GETTERS

PRÁCTICA 1 · GUIADA

Garaje de coches



Enunciado

Vas a programar un garaje donde se guardan coches.

Cada coche será un objeto creado con la clase Coche.

Al rellenar el formulario se crea un coche nuevo y aparece una tarjeta con sus datos.

Cada tarjeta tiene un botón para sumar 100 km al coche.



Pasos de ejecución

1. **Preparar la carpeta**
   - Copia `practica-01-garaje-de-coches` en `C:\laragon\www\js\clases\practicas\` y ábrela con Visual Studio Code.
2. **Completar script.js**
   - Lee el enunciado y los pasos de la cabecera. Escribe tu código bajo cada `TODO PASO`.
   - No modifiques `index.html` ni `estilos.css`.
3. **Guardar, abrir y verificar**
   - Guarda con `Ctrl + S` y comprueba que Apache está encendido en Laragon.
   - Abre `http://localhost/js/clases/practicas/practica-01-garaje-de-coches/index.html`.
   - Compara con las comprobaciones de esta práctica.

APOYO TEÓRICO

Clase y constructor
`class Coche { constructor(...) { ... } }`. El constructor guarda los datos con `this`. [1] [2]

Getter
`get antiguedad() { ... }` se lee como una propiedad: `coche.antiguedad`, sin paréntesis.

Instancias
`new Coche("Seat", "León", 2019)` crea un objeto nuevo con su propio km.



Práctica 1 (continuación)

PARTE 2 DE 3



Los pasos de la cabecera de script.js

Paso	Qué debes hacer (el detalle está en script.js)
1	Crea la clase Coche
2	Método describir()
3	Método recorrer(km)
4	Getter antiguedad
5	Crea el array coches
6	Función dibujar()
7	Escuchador del formulario

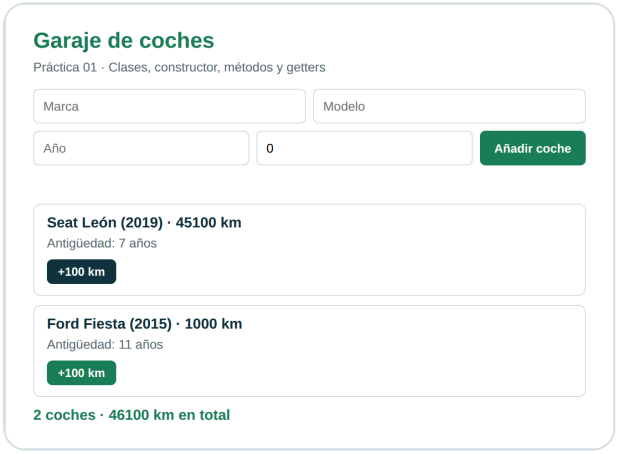


Qué clases debes crear

Clase	Propiedades	Métodos y getters
`Coche`	`marca, modelo, anio, km`	describir(), recorrer(km), get antiguedad



Lo que debes ver en el navegador



TRAS AÑADIR DOS COCHES Y PULSAR «+100 km»



Práctica 1 (continuación)

PARTE 3 DE 3



Qué debes comprobar

Acción	Resultado esperado
Añadir «Seat León 2019, 45000 km»	Tarjeta con «Seat León (2019) · 45000 km»
Pulsar «+100 km»	La tarjeta pasa a 45100 km y el resumen se actualiza
Año 1800	Mensaje «El año no es válido» y no se añade
Añadir un segundo coche	Dos tarjetas; el resumen suma los km de los dos



Errores frecuentes

Lo que ves	Qué debes revisar
`Class constructor Coche cannot be invoked without 'new'`	Falta `new` al crear el objeto.
`coche.describir is not a function`	Estás llamando al método sobre algo que no es un Coche. Revisa lo que guardas en el array.
`NaN` en los km	Convierte los valores del formulario con `Number()`.



Práctica 2

CAMPOS PRIVADOS, GETTERS Y ERRORES CONTROLADOS

PRÁCTICA 2 · GUIADA

Cuenta bancaria segura



Enunciado

Vas a programar una cuenta bancaria cuyo saldo no se puede tocar desde fuera.

Los datos internos serán campos privados (#) y solo se accederá a ellos mediante métodos y getters.

Los errores (PIN incorrecto, saldo insuficiente...) se lanzan con throw y se muestran con try...catch.



Pasos de ejecución

1 Preparar la carpeta

- Copia `practica-02-cuenta-segura` en `C:\laragon\www\js\clases\practicas\` y ábrela con Visual Studio Code.

2 Completar script.js

- Lee el enunciado y los pasos de la cabecera. Escribe tu código bajo cada `TODO PASO`.
- No modifiques `index.html` ni `estilos.css`.

3 Guardar, abrir y verificar

- Guarda con `Ctrl + S` y comprueba que Apache está encendido en Laragon.
- Abre `http://localhost/js/clases/practicas/practica-02-cuenta-segura/index.html`.
- Compara con las comprobaciones de esta práctica.

APOYO TEÓRICO

Campos privados
Se declaran con `#` dentro de la clase y solo se pueden usar dentro de ella. Fuera de la clase dan un error de sintaxis. [3]

throw y try...catch
`throw new Error("texto")` interrumpe el método. `try { ... } catch (error) { error.message }` lo captura.

Devolver copias
`[...this.#movimientos]` copia el array: quien lo recibe no puede alterar el original.



Práctica 2 (continuación)

PARTE 2 DE 3



Los pasos de la cabecera de script.js

Paso	Qué debes hacer (el detalle está en script.js)
1	Crea la clase CuentaSegura
2	Constructor
3	Método privado #registrar(texto)
4	Método ingresar(importe)
5	Método retirar(importe, pin)
6	Getters saldo e historial
7	Crea la cuenta y conecta los botones

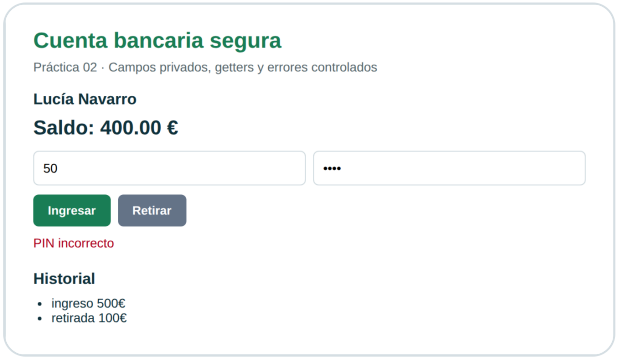


Qué clases debes crear

Clase	Propiedades	Métodos y getters
`CuentaSegura`	`#saldo, #movimientos, #pin, titular`	#registrar(), ingresar(), retirar(), get saldo, get historial



Lo que debes ver en el navegador



TRAS INGRESAR, RETIRAR Y UN PIN INCORRECTO



Práctica 2 (continuación)

PARTE 3 DE 3



Qué debes comprobar

Acción	Resultado esperado
Ingresar 500	Saldo 500.00 y movimiento «ingreso 500€»
Retirar 100 con PIN 1234	Saldo 400.00 y movimiento «retirada 100€»
Retirar con PIN 0000	Mensaje «PIN incorrecto»; el saldo no cambia
Retirar 9999	Mensaje «Saldo insuficiente»
Ingresar 0	Mensaje «Importe no válido»



Errores frecuentes

Lo que ves	Qué debes revisar
`Private field '#saldo' must be declared in an enclosing class`	Declara `#saldo = 0;` en el cuerpo de la clase, fuera del constructor.
El saldo no cambia en pantalla	Falta llamar a `pintar()` tras cada operación.
`cuenta.saldo = 500` no hace nada	Es lo esperado: `saldo` solo tiene getter.



Práctica 3

HERENCIA, SUPER, POLIMORFISMO Y MIEMBROS ESTÁTICOS

PRÁCTICA 3 · GUIADA

Figuras geométricas



Enunciado

Vas a crear una familia de figuras con herencia.

La clase Figura define lo común. Circulo y Rectangulo heredan de ella, y Cuadrado hereda de Rectangulo.

Todas las figuras responden a area() y perimetro(), pero cada una lo calcula a su manera (polimorfismo).

Un contador estático contará cuántas figuras se han creado.



Pasos de ejecución

1. **Preparar la carpeta**
   - Copia `practica-03-figuras-geometricas` en `C:\laragon\www\js\clases\practicas\` y ábrela con Visual Studio Code.
2. **Completar script.js**
   - Lee el enunciado y los pasos de la cabecera. Escribe tu código bajo cada `TODO PASO`.
   - No modifiques `index.html` ni `estilos.css`.
3. **Guardar, abrir y verificar**
   - Guarda con `Ctrl + S` y comprueba que Apache está encendido en Laragon.
   - Abre `http://localhost/js/clases/practicas/practica-03-figuras-geometricas/index.html`.
   - Compara con las comprobaciones de esta práctica.



Práctica 3 (continuación)

PARTE 2 DE 3



Los pasos de la cabecera de script.js

Paso	Qué debes hacer (el detalle está en script.js)
1	Clase Figura
2	Métodos abstractos
3	Método estático total()
4	Clase Circulo extends Figura
5	Clase Rectangulo extends Figura
6	Clase Cuadrado extends Rectangulo
7	Conecta el formulario

APOYO TEÓRICO

extends y super

`class Circulo extends Figura` hereda de Figura. En el constructor, `super(...)` va siempre antes de usar `this`. [2] [4]

Sobrescribir

Si la subclase define un método con el mismo nombre, se usa el suyo en lugar del del padre.

Estáticos

`static total()` se llama sobre la clase: `Figura.total()`, no sobre un objeto.



Qué clases debes crear

Clase	Propiedades	Métodos y getters
`Figura`	`nombre; static #contador`	area(), perimetro(), describir(), static total()
`Circulo extends Figura`	`radio`	area(), perimetro()
`Rectangulo extends Figura`	`ancho, alto`	area(), perimetro()
`Cuadrado extends Rectangulo`	`lado (usa ancho y alto)`	hereda todo



Práctica 3 (continuación)

PARTE 3 DE 3



Lo que debes ver en el navegador



TRAS AÑADIR LAS TRES FIGURAS



Qué debes comprobar

Acción	Resultado esperado
Círculo de radio 2	Círculo: área 12.57 · perímetro 12.57
Rectángulo 3 × 4	Rectángulo: área 12.00 · perímetro 14.00
Cuadrado de lado 5	Cuadrado: área 25.00 · perímetro 20.00
Con las tres figuras creadas	3 figuras creadas · área total: 49.57



Errores frecuentes

Lo que ves	Qué debes revisar
`Must call super constructor before accessing 'this'`	En un constructor de subclase, `super(...)` debe ir antes de cualquier `this`.
`Método no implementado`	Falta sobrescribir `area()` o `perimetro()` en la subclase.
`Figura.total is not a function`	El método estático necesita la palabra `static` delante.



Práctica 4

COLECCIONES DE OBJETOS, COMPOSICIÓN Y TOSTRING()

PRÁCTICA 4 · GUIADA

Biblioteca de libros



Enunciado

Vas a gestionar una biblioteca formada por libros.

La clase Libro representa cada ejemplar y la clase Biblioteca guarda la colección y gestiona los préstamos.

Se podrá buscar por título o autor, prestar y devolver libros.



Pasos de ejecución

1. **Preparar la carpeta**
   - Copia `practica-04-biblioteca` en `C:\laragon\www\js\clases\practicas\` y ábrela con Visual Studio Code.
2. **Completar script.js**
   - Lee el enunciado y los pasos de la cabecera. Escribe tu código bajo cada `TODO PASO`.
   - No modifiques `index.html` ni `estilos.css`.
3. **Guardar, abrir y verificar**
   - Guarda con `Ctrl + S` y comprueba que Apache está encendido en Laragon.
   - Abre `http://localhost/js/clases/practicas/practica-04-biblioteca/index.html`.
   - Compara con las comprobaciones de esta práctica.

APOYO TEÓRICO

Composición
Una Biblioteca TIENE libros: guarda objetos Libro dentro de un array privado. No hereda de Libro.

toString()
Si una clase define `toString()`, se usa al convertir el objeto a texto: `${libro}` o `libro.toString()`.

filter y find
`filter` devuelve los que cumplen la condición; `find` devuelve el primero (o `undefined`).



Práctica 4 (continuación)

PARTE 2 DE 3



Los pasos de la cabecera de script.js

Paso	Qué debes hacer (el detalle está en script.js)
1	Clase Libro
2	Método toString()
3	Clase Biblioteca
4	agregar(libro) y buscar(texto)
5	alternarPrestamo(id)
6	Getters total y disponibles
7	Conecta la página



Qué clases debes crear

Clase	Propiedades	Métodos y getters
`Libro`	`id, titulo, autor, anio, prestado; static #siguienteId`	toString()
`Biblioteca`	`#libros`	agregar(), buscar(), alternarPrestamo(), get total, get disponibles



Qué debes comprobar

Acción	Resultado esperado
Añadir tres libros	Tres tarjetas y «3 disponibles de 3 libros»
Pulsar «Prestar» en uno	La tarjeta pasa a «Prestado» y el resumen dice «2 disponibles de 3 libros»
Buscar «cer»	Solo se ven los libros de Cervantes
Pulsar «Devolver»	El libro vuelve a «Disponible»



Práctica 4 (continuación)

PARTE 3 DE 3



Lo que debes ver en el navegador



CON TRES LIBROS, UNO PRESTADO Y LA BÚSQUEDA «cer»



Errores frecuentes

Lo que ves	Qué debes revisar
`Cannot read properties of undefined (reading 'prestado')`	Estás usando `find` con un id que no existe. Comprueba el resultado antes de usarlo.
Se muestra `[object Object]`	Falta definir `toString()` en la clase Libro.
La búsqueda no encuentra nada	Pasa el texto a minúsculas en los dos lados de la comparación.



Práctica 5

CLASES, JSON, TOJSON(), STATIC Y LOCALSTORAGE

PRÁCTICA 5 · GUIADA

Gestor de tareas con JSON



Enunciado

Vas a programar un gestor de tareas que no pierde los datos al recargar la página.

La clase Tarea representa una tarea. La clase GestorTareas guarda la lista y la persiste en localStorage.

Para guardar y recuperar instancias necesitarás toJSON() y un método estático que las reconstruya.



Pasos de ejecución

1. **Preparar la carpeta**
   - Copia `practica-05-gestor-de-tareas` en `C:\laragon\www\js\clases\practicas\` y ábrela con Visual Studio Code.
2. **Completar script.js**
   - Lee el enunciado y los pasos de la cabecera. Escribe tu código bajo cada `TODO PASO`.
   - No modifiques `index.html` ni `estilos.css`.
3. **Guardar, abrir y verificar**
   - Guarda con `Ctrl + S` y comprueba que Apache está encendido en Laragon.
   - Abre `http://localhost/js/clases/practicas/practica-05-gestor-de-tareas/index.html`.
   - Compara con las comprobaciones de esta práctica.

APOYO TEÓRICO

toJSON()
Si una clase tiene `toJSON()`, `JSON.stringify` lo usa para decidir qué datos se guardan. [5]

Método estático de reconstrucción
JSON.parse devuelve objetos planos, sin métodos. `Tarea.desdeJSON(datos)` crea de nuevo instancias de Tarea.

localStorage
Solo guarda texto. Si todavía no hay datos, `getItem` devuelve `null`: usa `|| []`. [6]



Práctica 5 (continuación)

PARTE 2 DE 3



Los pasos de la cabecera de script.js

Paso	Qué debes hacer (el detalle está en script.js)
1	Clase Tarea
2	Método alternar()
3	toJSON() y static desdeJSON(datos)
4	Clase GestorTareas
5	Métodos agregar, alternar, eliminar y filtrar
6	Getter pendientes
7	Conecta la página



Qué clases debes crear

Clase	Propiedades	Métodos y getters
`Tarea`	`id, texto, prioridad, hecha`	alternar(), toJSON(), static desdeJSON()
`GestorTareas`	`#tareas, #clave`	#guardar(), #cargar(), agregar(), alternar(), eliminar(), filtrar(), get pendientes



Qué debes comprobar

Acción	Resultado esperado
Añadir tres tareas	Tres tarjetas y «3 pendientes de 3»
Pulsar «Hecha» en una	La tarjeta muestra «Hecha» y el resumen pasa a «2 pendientes de 3»
Filtro «Pendientes»	Solo se ven las dos tareas sin completar
Recargar la página	Las tareas siguen ahí, con su estado
Pulsar «Borrar»	La tarea desaparece y el cambio se guarda



Práctica 5 (continuación)

PARTE 3 DE 3



Lo que debes ver en el navegador



TRAS AÑADIR TRES TAREAS Y MARCAR UNA



Errores frecuentes

Lo que ves	Qué debes revisar
`tarea.alternar is not a function` tras recargar	Cargaste objetos planos. Reconstruye cada uno con `Tarea.desdeJSON`.
`Unexpected token` al arrancar	El texto guardado no es JSON válido: ejecuta `localStorage.clear()` en la consola.
Dos tareas con el mismo id	`Date.now()` puede repetirse si añades varias tareas en el mismo milisegundo. Genera un id único.



Práctica 6

RETO FINAL: HERENCIA, COMPOSICIÓN, GETTERS Y ESTÁTICOS

PRÁCTICA 6 · RETO FINAL

Tienda con carrito



Enunciado

Vas a programar una pequeña tienda con dos tipos de producto.

Los productos físicos tienen peso y gastos de envío. Los productos digitales no se envían.

El carrito está formado por líneas (producto + cantidad) y calcula subtotal, envío y total.

Es el reto final: reúne herencia, composición, getters y campos privados.



Pasos de ejecución

1. **Preparar la carpeta**
   - Copia `practica-06-tienda-con-carrito` en `C:\laragon\www\js\clases\practicas\` y ábrela con Visual Studio Code.
2. **Completar script.js**
   - Lee el enunciado y los pasos de la cabecera. Escribe tu código bajo cada `TODO PASO`.
   - No modifiques `index.html` ni `estilos.css`.
3. **Guardar, abrir y verificar**
   - Guarda con `Ctrl + S` y comprueba que Apache está encendido en Laragon.
   - Abre `http://localhost/js/clases/practicas/practica-06-tienda-con-carrito/index.html`.
   - Compara con las comprobaciones de esta práctica.

APOYO TEÓRICO

Herencia y composición
ProductoFisico ES un Producto (herencia). Un Carrito TIENE líneas y cada línea TIENE un producto (composición). [4]

super.metodo()
`super.describir()` ejecuta el método del padre dentro de la subclase.

Fábrica de objetos
Una función como `crearProducto(datos)` decide qué clase usar según los datos.



Práctica 6 (continuación)

PARTE 2 DE 3



Los pasos de la cabecera de script.js

Paso	Qué debes hacer (el detalle está en script.js)
1	Clase Producto
2	Clase ProductoFisico extends Producto
3	Clase ProductoDigital extends Producto
4	Clase LineaCarrito
5	Clase Carrito
6	Getters del Carrito
7	Conecta la página



Qué clases debes crear

Clase	Propiedades	Métodos y getters
`Producto`	`nombre, precio`	costeEnvio(), describir()
`ProductoFisico extends Producto`	`peso`	costeEnvio() sobrescrito
`ProductoDigital extends Producto`	—	describir() sobrescrito con super
`LineaCarrito`	`producto, cantidad`	get subtotal, get envio
`Carrito`	`#lineas`	agregar(), quitar(), vaciar(), get lineas, subtotal, envio, total



Qué debes comprobar

Acción	Resultado esperado
Añadir el teclado dos veces	Una línea «x 2 · 119.80 €»
Añadir el curso de JavaScript	Una línea digital «x 1 · 29.00 €»
Con ambos en el carrito	Subtotal 148.80 · Envío 4.80 · Total 153.60
Pulsar «Quitar» en el teclado	Solo queda el curso y el envío pasa a 0.00
Pulsar «Vaciar carrito»	El carrito queda vacío con todo a 0.00

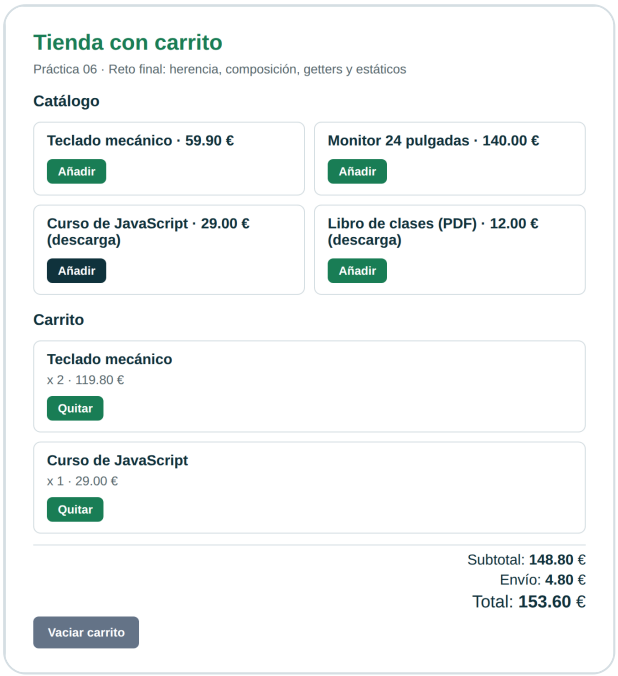


Práctica 6 (continuación)

PARTE 3 DE 3



Lo que debes ver en el navegador



CON EL TECLADO ×2 Y EL CURSO EN EL CARRITO



Errores frecuentes

Lo que ves	Qué debes revisar
`producto.costeEnvio is not a function`	Todos los productos deben tener `costeEnvio()`: defínelo en la clase padre.
El envío de los digitales no es 0	No sobrescribas `costeEnvio()` en `ProductoDigital`: hereda el del padre.
Las líneas se duplican	En `agregar`, busca antes si ya existe una línea con ese producto.



Fuentes y Documentación Oficial

REFERENCIAS DE LAS PRÁCTICAS

Cada número entre corchetes remite a la documentación correspondiente.



Fuentes y Documentación Oficial

[1] Clases · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Classes

[2] constructor · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes/constructor

[3] Private elements · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Classes/Private_elements

[4] extends · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Classes/extends

[5] JSON.stringify() · MDN Web Docs · developer.mozilla.org/JavaScript/Reference/Global_Objects/JSON/stringify

[6] Window.localStorage · MDN Web Docs · developer.mozilla.org/es/docs/Web/API/Window/localStorage